

WEEK – 4, 5

Introduction to Classes, objects, constructors, methods, parameter passing, overloading constructors and methods.

Sample Program:

3. Classes and Objects CO1

Design a “farm animals” java application with the details of animals like cow, pig, horse.

Consider the following details like where they stay, what they eat, the sound they make by

using classes and objects. 4. Write a Java Program to below class diagram: CO1

5. Constructor overloading CO1

An organization is maintaining the data of employee according to cadre of employee with following parameters: name, id, designation, salary, promotion status. Apply the constructor overloading to implement it.

Steps of the Code

Step 1: Create the Animal class

The Animal class contains three private variables:

name
habitat
food

These represent the common properties of animals.

Step 2: Create the constructor

The constructor initializes these values:

Animal(String name, String habitat, String food)

Step 3: Create getter methods

The methods:

getname()
gethabitat()
getfood()

are used to access the private data.

This demonstrates encapsulation.

Step 4: Create the sound() method

The parent class has:

```
String sound() {  
    return "animal sound";  
}
```

This provides a general animal sound.

Step 5: Create the show() method

The show() method displays:

Animal name
Habitat
Food
Sound

Step 6: Create the cow class

class cow extends Animal

cow inherits from Animal.

Its constructor calls:

```
super("cow", "shed", "grass");
```

The cow overrides sound():

moo

Step 7: Create the pig class

The pig inherits from Animal.

Its information is:

Name → pig

Habitat → sty

Food → grains,vegetable

Sound → oink

Step 8: Create the horse class

The horse inherits from Animal.

Its information is:

Name → horse

Habitat → stable

Food → grass

Sound → neigh

Step 9: Create objects in main()

Three objects are created:

```
horse farm = new horse();
```

```
pig far = new pig();
```

```
cow fa = new cow();
```

Step 10: Call show()

Each object's show() method displays its own details
and its overridden sound.

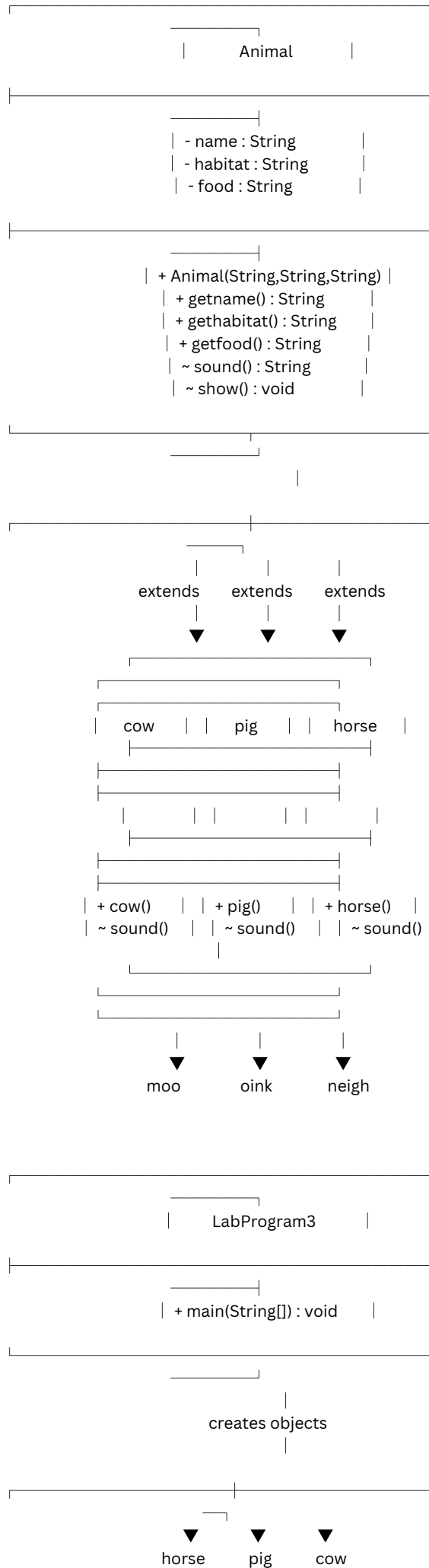
Step 11: End the program

The program finishes after displaying all three animals.

START

1. Define the Animal class.
2. Declare private variables:
 name
 habitat
 food
3. Create the Animal constructor to initialize
 name, habitat and food.
4. Create getter methods to access the private
 variables.
5. Create sound() method in Animal that returns
 "animal sound".
6. Create show() method to display:
 animal name
 habitat
 food
 sound
7. Create cow class by extending Animal.
 Set name = cow
 Set habitat = shed
 Set food = grass
 Override sound() to return "moo".
8. Create pig class by extending Animal.
 Set name = pig
 Set habitat = sty
 Set food = grains,vegetable
 Override sound() to return "oink".
9. Create horse class by extending Animal.
 Set name = horse
 Set habitat = stable
 Set food = grass
 Override sound() to return "neigh".
10. In main():
 Create horse object.
 Display horse details.
11. Create pig object.
 Display pig details.
12. Create cow object.
 Display cow details.

13. STOP



```

        class Animal{
            private String name;
            private String habitat;
            private String food;
            public Animal(String name , String habitat , String food)
            {
                this.name = name;
                this.habitat = habitat;
                this.food = food;
            }
            public String getname(){
                return name;
            }
            public String gethabitat(){
                return habitat;
            }
            public String getfood(){
                return food;
            }
            String sound(){
                return "animal sound";
            }
            void show(){

                System.out.println("animal is " + name);
                System.out.println("habitat is" + habitat);
                System.out.println("food is " + food);
                System.out.println("sound is " + sound());
                System.out.println("-----");

            }

        }

        class cow extends Animal{
            public cow(){
                super("cow", "shed", "grass");
            }
            @Override
            String sound(){
                return "moo";
            }
        }

        class pig extends Animal{
            public pig(){
                super("pig","sty","grains,vegetable");
            }
            @Override
            String sound(){
                return "oink";
            }
        }

        class horse extends Animal{
            public horse(){
                super("horse" , "stable", "grass");
            }
            @Override
            String sound(){
                return "neigh";
            }
        }

        public class LabProgram3 {
            public static void main(String[] args) {
                horse farm = new horse();
                farm.sound();
                farm.show();
                pig far = new pig();
                farm.sound();
                far.show();
                cow fa = new cow();
                fa.sound();
                fa.show();

            }
        }

```

output of code

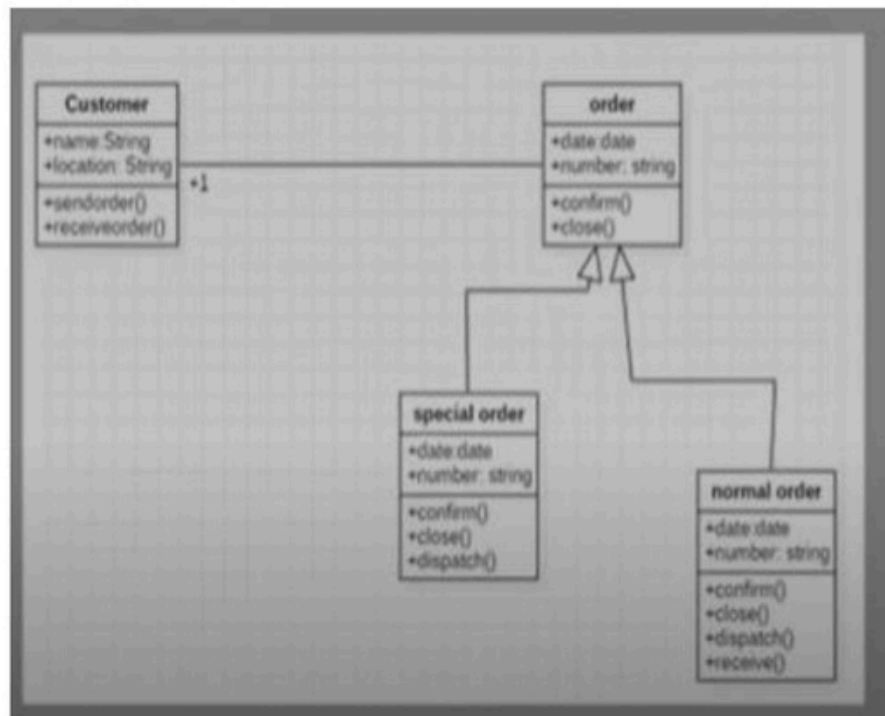
```
javac LabProgram3.java  
java LabProgram3
```

```
    animal is horse  
    habitat is stable  
    food is grass  
    sound is neigh
```

```
    animal is pig  
    habitat is sty  
food is grains,vegetable  
    sound is oink
```

```
    animal is cow  
    habitat is shed  
    food is grass  
    sound is moo
```

4. Write a Java Program to below class diagram:



Steps of the Code

Step 1: Create the order class

The order class contains two private variables:

date
number

These represent the order date and order number.

Because they are private, this demonstrates encapsulation.

Step 2: Create the order constructor

The constructor initializes:

date
number

using:

this.date = date;
this.number = number;

Step 3: Create confirm() method

confirm()

displays the order number and confirmation date.

Step 4: Create close() method

close()

displays that the order has been closed.

Step 5: Create specialorder

class specialorder extends order

This means specialorder inherits from order.

It has an additional method:

dispatch()

which displays that the special order has been

dispatched.

Step 6: Create normalorder

class normalorder extends order

This class also inherits from order.

It has two additional methods:

dispatch()
receive()

Step 7: Create customer

The customer class has:

name
location

Both are private.

It contains:

sendorder()
receiveorder()

Step 8: sendorder() receives an order

public void sendorder(order order)

Notice that the parameter type is order.

Therefore, an object of a child class such as specialorder can also be passed.

Step 9: receiveorder() receives an order

void receiveorder(order order)

It displays that the customer received the order and

then calls:

order.close();

Step 10: Create customer object

customer c = new customer("hemanth", "vizag");

Step 11: Create special order

specialorder so =

new specialorder("12th august", "44");

Step 12: Send special order

c.sendorder(so);

The customer sends the special order.

Step 13: Dispatch special order

so.dispatch();

Step 14: Receive special order

c.receiveorder(so);

The order is received and then closed.

Step 15: Create normal order

normalorder no =

new normalorder("13th august", "82");

Step 16: Process normal order

The program:

Receives the order.

Closes the order.

Dispatches the order.

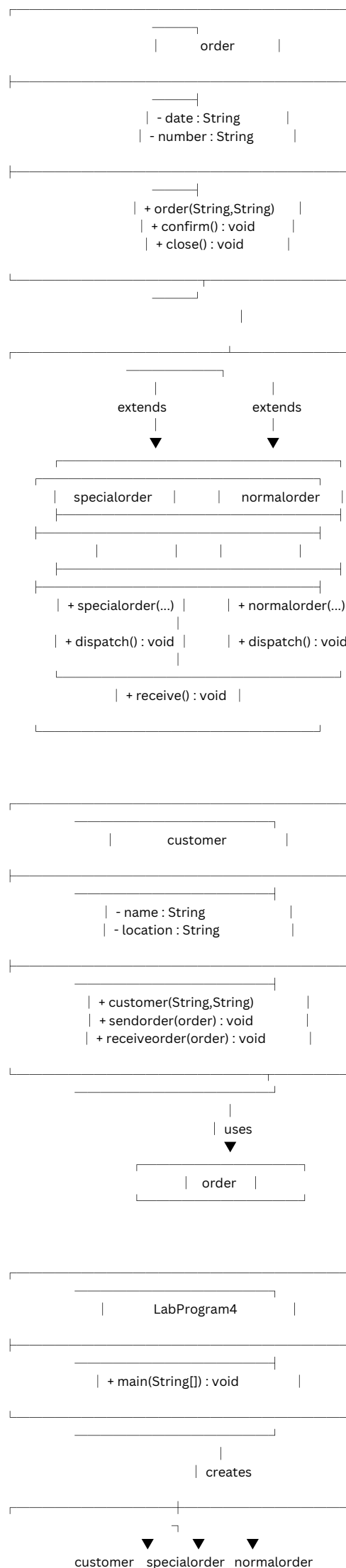
Receives it.

Calls receiveorder() again.

START

1. Define the Order class.
2. Declare private variables:
 date
 number
3. Create the Order constructor to initialize
 date and number.
4. Define confirm() to display order confirmation.
5. Define close() to display that the order is closed.
6. Create SpecialOrder class by extending Order.
7. Create SpecialOrder constructor and call the
 parent constructor using super(date, number).
8. Define dispatch() for special orders.
9. Create NormalOrder class by extending Order.
10. Create NormalOrder constructor and call the
 parent constructor using super(date, number).
11. Define dispatch() and receive() for normal orders.
12. Create Customer class.
13. Declare private variables:
 name
 location
14. Create Customer constructor.
15. Define sendorder() to send an order.
16. Define receiveorder() to receive and close an order.
17. In main(), create a Customer object.
18. Create a SpecialOrder object.
19. Send the special order.
20. Dispatch the special order.
21. Receive and close the special order.
22. Create a NormalOrder object.
23. Receive and close the normal order.
24. Dispatch the normal order.
25. Receive the normal order.
26. Receive and close the normal order again.

27. STOP



```

        class order{
            private String date;
            private String number;

            public order(String date , String number){
                this.date = date;
                this.number = number;
            }

            public void confirm(){
                System.out.println("order" + number + "confirm on "+
                    date);
            }

            public void close(){
                System.out.println("order " + number + "closed");
            }
        }

        class specialorder extends order{
            public specialorder(String date , String number){
                super("date" , "number");
            }

            public void dispatch(){
                System.out.println("specialorder is " + "dispatched");
            }
        }

        class normalorder extends order{
            public normalorder(String date , String number){
                super("date" , " number");
            }

            public void dispatch(){
                System.out.println("normalorder is " + "dispatched");
            }

            public void receive(){
                System.out.println("normalorder is" + "received by
                    customer");
            }
        }

        class customer{
            private String name;
            private String location;
            public customer(String name , String location){
                this.name = name;
                this.location = location;
            }

            public void sendorder( order order){
                System.out.println(name + "from" + location + "sent
                    an order");
            }

            void receiveorder(order order){
                System.out.println(name + "received order");
                order.close();
            }
        }

        public class LabProgram4 {
            public static void main(String[] args) {

                customer c = new customer("hemanth", "vizag");
                specialorder so = new specialorder("12th august", "44");
                c.sendorder(so);
                so.dispatch();
                c.receiveorder(so);
                System.out.println();
                normalorder no = new normalorder("13th august", "82");
                c.receiveorder(no);
                no.dispatch();
                no.receive();
                c.receiveorder(no);
            }
        }

```

Output of code

```
javac LabProgram4.java
```

```
java LabProgram4
```

```
hemanth from vizag sent an order  
special order is dispatched  
hemanth received order  
order 44 closed
```

```
hemanth received order  
order 82 closed  
normal order is dispatched  
normal order is received by customer  
hemanth received order  
order 82 closed
```

Constructor overloading

An organization is maintaining the data of employee according to cadre of employee with following parameters: name, id, designation, salary, promotion status. Apply the constructor overloading to implement it.

Steps of the Program

Step 1: Create the Employee class

The class contains five private data members:

name

id

designation

salary

promotionstatus

Since they are private, they cannot be directly accessed from outside the class.

This demonstrates data hiding/encapsulation.

Step 2: Create the default constructor

Employee()

It assigns default values:

name = not assigned

id = 0

designation = trainer

salary = 0.0

promotionstatus = false

Step 3: Create a 2-parameter constructor

Employee(String name, int id)

It accepts:

name

id

The other values receive default values.

Step 4: Create a 4-parameter constructor

Employee(String name, int id,

String designation, double salary)

It accepts four employee details.

promotionstatus is set to false by default.

Step 5: Create a 5-parameter constructor

Employee(String name, int id,

String designation, double salary,

boolean promotionstatus)

This accepts all five employee details.

Step 6: Create show() method

The show() method displays all employee details.

Step 7: Create employee objects

Four objects are created:

e1 → default constructor

e2 → 2-parameter constructor

e3 → 4-parameter constructor

e4 → 5-parameter constructor

Step 8: Display employee details

The show() method is called for every object.

Step 9: End

The program terminates.

START

1. Define the Employee class.

2. Declare private variables:

name
id
designation
salary
promotionstatus

3. Define the default constructor.

Initialize:
name = "not assigned"
id = 0
designation = "trainer"
salary = 0.0
promotionstatus = false

4. Define a constructor with 2 parameters:

name
id

Assign default values to the remaining
attributes.

5. Define a constructor with 4 parameters:

name
id
designation
salary

Set promotionstatus = false.

6. Define a constructor with 5 parameters:

name
id
designation
salary
promotionstatus

7. Define show() method to display
all employee details.

8. In main():

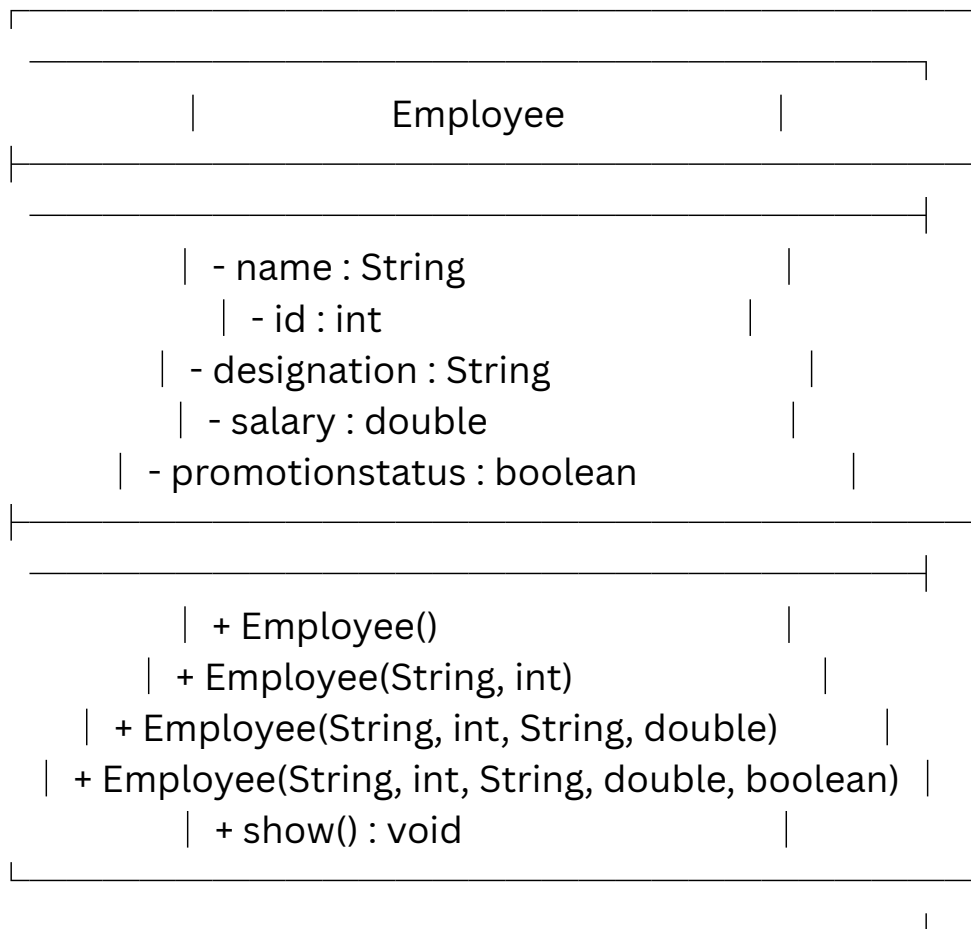
- a. Create employee e1 using default
constructor.
- b. Display e1 details.

9. Create employee e2 using 2-parameter
constructor.
Display e2 details.

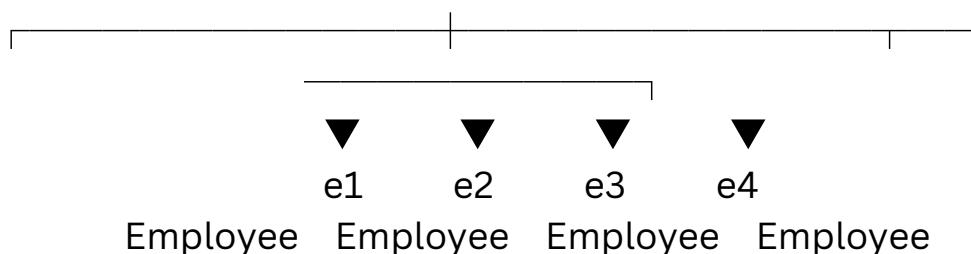
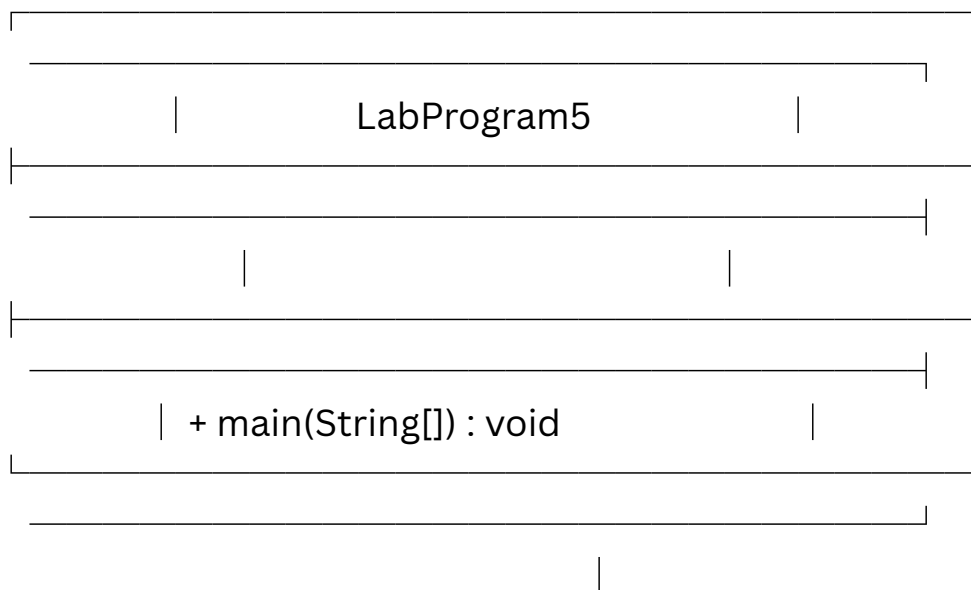
10. Create employee e3 using 4-parameter
constructor.
Display e3 details.

11. Create employee e4 using 5-parameter
constructor.
Display e4 details.

12. STOP



creates



```

        class Employee{
            private String name;
            private int id;
            private String designation;
            private double salary;
            private boolean promotionstatus;
            // constructor is default const is create
            public Employee(){
                this.name = "not assigned";
                this.id = 0;
                this.designation = "trainer";
                this.salary = 0.0;
                this.promotionstatus = false;
            }
            // const have passes 2 parameters by using const
            // overloading
            public Employee(String name , int id){
                this.name = name;
                this.id = id;
                this.designation = "trainer";
                this.salary = 0.0;
                this.promotionstatus = false;
                System.out.println("--- 2 parameters passing
                constructor");
            }

            // const have passes 4 parameters

            public Employee(String name , int id , String designation
            , double salary){
                this.name = name;
                this.id = id;
                this.designation = "trainer";
                this.salary = 0.0;
                this.promotionstatus = false;
                System.out.println("4 parameters passing constructor");
            }

            public Employee(String name , int id , String designation
            , double salary , boolean promotionstatus){
                this.name = name;
                this.id = id;
                this.designation = "trainer";
                this.salary = 0.0;
                this.promotionstatus = false;
                System.out.println("all details passing constructor");
            }

            public void show(){
                System.out.println("--- employee details ---");
                System.out.println("name : " + name);
                System.out.println("id : " + id);
                System.out.println("designation : " + designation);
                System.out.println("salary : " + salary);
                System.out.println("promotionstatus : " +
                promotionstatus);
            }
        }

        public class LabProgram5 {

            public static void main(String[] args) {
                Employee e1 = new Employee();
                e1.show();
                Employee e2 = new Employee("hemanth" , 244);
                e2.show();
                Employee e3 = new Employee("hemanth" , 244 ,
                "software engineer" , 70.000);
                e3.show();
                Employee e4 = new Employee("hemanth" , 244 ,
                "software engineer" , 75.000 , false);
                e4.show();
            }
        }

```

output of code

```
javac LabProgram5.java  
java LabProgram5
```

```
--- employee details ---  
    name : not assigned  
        id : 0  
    designation : trainer  
        salary : 0.0  
    promotionstatus : false  
  
--- 2 parameters passing constructor ---  
    --- employee details ---  
        name : hemanth  
            id : 244  
        designation : trainer  
            salary : 0.0  
        promotionstatus : false  
  
--- 4 parameters passing constructor ---  
    --- employee details ---  
        name : hemanth  
            id : 244  
    designation : software engineer  
        salary : 70.0  
    promotionstatus : false  
  
--- all details passing constructor ---  
    --- employee details ---  
        name : hemanth  
            id : 244  
    designation : software engineer  
        salary : 75.0  
    promotionstatus : false
```